Qualifying Exam Report

Muhammad Tehmasib Ali Tashfeen
Doctoral Student, School of Computing, Wichita State University

Submitted to the School of Computing
and the faculty of the Graduate School of
Wichita State University
in partial fulfillment of
the requirements for the degree of
Doctor of Philosophy in Electrical Engineering and Computer Science

October 15, 2025

TABLE OF CONTENTS

Chapter	Page
1 Introduction	1
2 JLeaks: A Featured Resource Leak Repository Collected From Hundreds of Open Source Java Projects [1]	2
3 Large Language Models for Software Engineering: A Systematic Literature Review [2]	7
BIBLIOGRAPHY	14

LIST OF FIGURES

Figure		Page
1	Rules for filtering projects and commits.	3
2	The workflow of building JLeaks	5
3	Cohen's Kappa metric.	5
4	State-of-the-art surveys related to LLMs for SE.	8
5	Study identification and selection process.	9
6	Inclusion criteria and Exclusion criteria.	10
7	Checklist of Quality Assessment Criteria (QAC) for LLM studies in SE.	11
8	Overview of the selected 395 papers' distribution.	12

CHAPTER I

Introduction

Imagine there is an attack vector in every home, organization, and police station, and the list goes on, and in the hands of every person. The device is portable and stores critical data and supports a variety of technologies. The device is susceptible to numerous software and hardware exploitations. Unfortunately that device is a mobile phone. Mobile phones constitute the biggest attack vector of our society, and there are fewer ways to secure them. People depend more on mobile devices than ever before; we know that, and so do the bad actors. According to the World Economic Forum, there are approximately 8.5 billion mobile devices, which are more than people in the world [3]. There are two famous mobile operating systems (OS): Android [4] and iOS [5].

Now a days attackers are not just throwing viruses to the users, they are creating ecosystems to compromise confidential data. Research reported by Malwarebytes indicates 151% rise in malware attacks on android devices on the first half of 2025 [6]. Modern android applications are not just Java code, they contain diverse third-party frameworks, automated built tools, and continuous delivery (CD) pipelines that simply expands the attack vectors. The complex secure software development lifecycle (SSDLC), resource management and development constraints remains a challenging task to secure a android application. One of the major issues in android development is memory leakage, especially related to GDPR or Google data safety policies that are associated to user privacy.

The Android Operating System (OS) is the most widely used mobile operating system that powers billions of smartphones and processes large amounts of confidential data [7]. Maintaining the integrity and confidentiality of this platform is crucial. The Android Runtime (ART) primarily manages memory efficiently through its state-of-the-art Garbage Collection (GC) [8]. This runtime environment provides advantages over native code by automatically managing the memory allocation and deallocation, thus reducing the risks of

traditional memory corruption that are common in C and C++ [7].

The ART relies on Android application developers following component life cycles and precisely maintaining object references. Memory leak will occur if allocated memory that is no longer needed is not released. This issue affects the application reliability and performance [9]. Inadequate practices, such as negligence to dispose of resources or ignoring to unregister listeners, cause objects to unintentionally occupy memory that prevents the GC from cleaning the resources [10]. Over time, these unreleased objects increased, causing the system to degrade its performance or crash due to drained resources, which are called Denial of Service (DoS) incidents [11].

My area of subject is mobile security, specifically how to secure Android applications so that they don't leak information. One of the less studied areas of memory and resource leaks is in the domain of Android applications. The reason for these types of violations is when developers unintentionally abuse an Android API, don't follow lifecycle requirements, or poorly manage object references, especially long-lived objects, e.g., `Context`, `Activity`, or `Fragment`. My research work will consist of three objectives. First, to create a dataset from real-world applications that is annotated, highly available, consistent, and unique. Second, to create a novel static analysis tool to detect certain memory or resource leakage in Android applications. Third, to use LLMs or RAG to enhance the detection, localization, and fixtures of defected code in the application program.

JLeaks: A Featured Resource Leak Repository Collected From Hundreds of Open Source Java Projects [1]

Introduction

Improper resource management causes memory and resource leaks in programs, which leads to degraded system performance or even causes internal denial of services (DoS), resulting in system-wide crashes. It is a challenge to identify these defects in the early stage of software application development because they are triggered only when the resource allocation is

combined to a certain extent. Memory leaks happen when objects in the program retain memory, while resource leaks happen when system resources like files, sockets, and threads are retained after use.

The works in [1] created a dataset that would help to better design and validate resource leakage detection, localization, and the creation of patching tools. Further, it can be utilized to test existing algorithms. The study aimed to resolve the three main issues for the creation of the data set. First, the high-quality repository containing leaks from real-world projects is too small to build machine learning models. Second, code duplication is noted in a large number of existing software projects, which leads to a limited study of software applications. Third, the access and availability of some software projects decreases over time.

In the paper, they introduced a resource leak dataset collected from real-world projects from GitHub. The authors defined predefined rules to create and find high-quality Java projects as depicted in figure 1. To further enhance the accuracy, they run a two-round manual cross-validation. For ease of use, they successfully compiled some of the projects into bytecode. This results in the largest and most detailed resource leakage dataset so far, which they named JLeaks. The dataset provides GitHub commit links, source files, patches, and other relevant information.

Rules for project	Keywords for commits	
	Message	Code
# of commits > 20	leak	close
# of issues > 20	leakage	stop
# of contributors > 10	close	shutdown
# of releases > 0	closed	try(*)
# of stars > 500	release	CloseUtil
duration > 50 weeks	resource	-

Figure 1: Rules for filtering projects and commits.

Contributions

They used semi-automated workflow to mine real-world resource leaks from GitHub. They reduce the man-hour costs by applying filters, keywords, and automatic compilation. The dataset contains 1,094 resource leaks from 321 open-source projects. More than 3,185 commits were filtered and validated before they successfully built JLeaks. Each bug is tagged and includes defect code, patches, key variables, and location. This provides defect code analysis, automated testing, and tool evaluation. They compared Jleaks with DriodLeaks [12] and Leak19 [13]. The comparison shows that JLeaks is more detailed, highly available, unique, and consistent. In addition to that, they used JLeaks to evaluate existing tools to summarize their strengths and weaknesses.

Approach

The workflow of creating the dataset is illustrated in figure 2. It has three steps: mining the data, tagging the data, and compiling. The JLeaks repository is hosted on GitHub[1]. The GitHub platform is a well-known platform for managing the open-source projects. Individual commits have a *diff code* and a message. The message has a description of changes made by the programmer. *Diff code* represents the difference in code before the pre-patch and after the patch. The code with the "+" indicates patched code, while "-" represents defected code. The authors selected desired projects from the GitHub platform to ensure accuracy and reliability. The projects were selected according to the rules as specified in figure 1. Then they perform two methods to analyze the commits by programmers. They filter commit messages using the keywords in the second column of figure 1. They also created some keywords for code as shown in the third column of figure 1. If the pre-patched or patched code contains any words from the third column of figure 1, that commit more likely fixed the resource leakage.

[1]https://github.com/Dcollectors/JLeaks

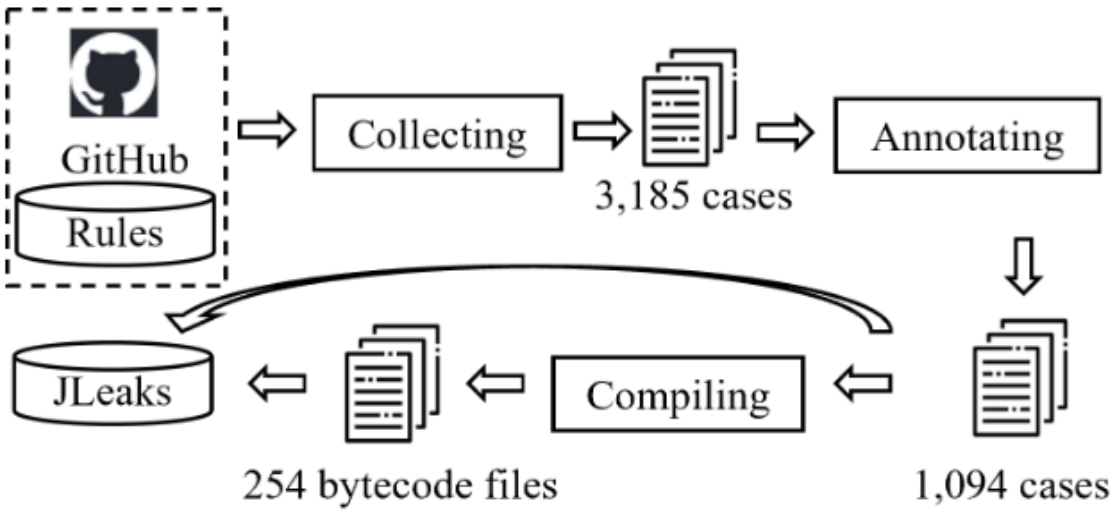


Figure 2: The workflow of building JLeaks.

In order to reduce the number of false positives, they carefully investigate commits and the diff code. Cross validation and manual work were done with the help of five experienced Java developers from the industry. Those developers have at least 8 years of experience. Three students also examined the dataset afterwards. If conflicts arise, they conduct a joint meeting to discuss issues. In total they successfully collected 1,094 defects from 321 projects. They used Cohen's Kappa metric, as shown in figure 3, to calculate the consistency cross validation.

Dimension	Cohen's Kappa
resource leak	0.88
resource type	0.74
root cause	0.74
fix approach	0.92

Figure 3: Cohen's Kappa metric.

Using the source code to detect the bugs has been explored by the authors extensively. Popular bug detection tools mostly rely on bytecode files, e.g., SpotBugs [14] and Infer [15].

The reason for using bytecode is sometimes the source code is incomplete or unavailable. The authors first try to compile all the projects. But due to their legacy nature, they required outdated dependencies. Only 86 projects are successfully compiled on default configurations. Then they created an automatic warper to compile the defected files, which iterates on building tools to enumerate the remote Maven repositories. At the end only 254 defects were complied with in their source files and bytecodes.

Limitations

It was a challenge to build a fully automated end-to-end toolchain. Even though the commit messages explicitly mention "resource leak," etc., there are still duplicate and garbage data. The authors collected 1,094 resource leaks from open-source projects using a semi-automatic toolchain. Approximately 900 man-hours were spent to annotate and verify the dataset of JLeaks. It was time-consuming and costly. The authors find in their study that a large number of resource leaks are due to the programmer's lack of awareness to prevent resource leakage. Most resource leakage was not intra-procedural in nature, according to their statistical analysis. The automated compilation of open-source projects is challenging. Even with their best efforts, they successfully compiled 86 versions of projects, and 254 bytecode files were generated in correspondence to the defects. Finally, in their findings, they suggest that tools that detect resource leaks should focus more on intra-procedural resource leak checks. They observed that most resource leaks are intra-procedural. They also noted that resource leaks related to third-party libraries are often overlooked during defect detection, which results in a high rate of false negatives.

Why it is relevant to our research?

One of the objectives of our research is to build a high-quality dataset that consists of resource or memory leakages from Android applications. It will help to design and train machine learning models. Static analysis tools can also be created or benchmarked using the dataset.

Large Language Models for Software Engineering: A Systematic Literature Review [2]

Introduction

Traditional Language Models (LMs) are foundational for Large Language Models (LLMs). LMs based the foundations and developed the basis for generating text and understanding. Increased computational power, advanced machine learning algorithms, and ease of access to large-scale data lead to the emergence of working LLMs. Human-like linguistic capabilities were developed due to the expensive, diverse nature of training data. The authors noticed a blurred line between machine-generated language and human language.

Software Engineering (SE) is a discipline that develops, implements, and maintains software applications. SE is taking full advantage of the LLM revolution [16]. Traditional SE challenging tasks are now reframed in LLMs for code, data, and text analysis [17]. Using LLMs to solve those SE challenges has proven to be so advantageous [18, 19, 20]. An LLM named Codex solved approximately 72.31% of complex tasks in the Python programming language that are posed by humans [21]. ChatGPT-4 from OpenAI has strong performance in code generation, understanding, and reasoning [22]. Figure 4 shows the summary of the literature review or surveys on LLM-related work. The work in 4 has limitations. They either focus narrowly on a single subject of SE, e.g., software testing using LLMs [23] and natural language to code (NL2Code) [24], or they are focused on machine learning (ML) or deep learning (DL) models. Those studies overlooked more advanced and current LLM applications such as ChatGPT, which has emerging applications in the SE field [25, 26]. On top of that, those studies merely looked into the performance of LLMs in SE tasks. The authors in [2] conducted a Systematic Literature Review (SLR) on the utilization of LLMs in SE (LLM4SE), which bridges the gap between SE and LLMs.

Reference	Year	Scope of models[1]	Scope of SE tasks	SLR[2]	Time frame	# Collected Papers
Zhang *et al.* [543]	2023	Code LLM	Automated program repair	✓	2017-2023	185
Zheng *et al.* [564]	2023	Code LLM	General SE scope	✓	2021-2023	149
Fan *et al.* [85]	2023	LLM	General SE scope	×	-	Not specified
Zan *et al.* [526]	2023	LLM (12M+)	NL2Code	×	2020-2023	Not specified
Wang *et al.* [448]	2023	LLM (117M+)	Software testing	✓	2019-2023	52
Wang *et al.* [452]	2022	ML, DL[3]	General SE scope	✓	2009-2020	1,209 (ML) + 358 (DL)
Yang *et al.* [509]	2022	DL	General SE scope	✓	2015-2020	250
Watson *et al.* [466]	2022	DL	General SE scope	✓	2009-2019	128
Our work	2024	LLM	General SE scope	✓	2017-2024	395

[1] "M" means million and "B" means billion. The numbers in parentheses indicate the parameter sizes of LLMs.
[2] SLR stands for Systematic Literature Review. This column denotes whether the paper follows an SLR process.
[3] ML and DL refer to Machine Learning and Deep Learning, respectively.

Figure 4: State-of-the-art surveys related to LLMs for SE.

Contributions

The research involved a comprehensive Systematic Literature Review (SLR), making it the first to thoroughly analyze 395 papers published between January 2017 and January 2024 covering the application of LLMs to solve problems in SE. The study provided a detailed breakdown of publication patterns and the types of LLMs being utilized across the domain. Furthermore, it meticulously described the complete data lifecycle, from collection and pre-processing to representation, and examined the optimization strategies researchers are using, including prompt engineering techniques and performance evaluation metrics. Crucially, the review mapped out the practical utility of LLMs in SE by classifying their use into a wide array of 85 specific tasks spanning the six main phases of the software development life cycle—from initial requirements to ongoing maintenance and management.

Approach

This study followed the well-established Systematic Literature Review (SLR) methodology proposed by Kitchenham and colleagues, a widely recognized framework in Software Engineering research. The review process was structured into three main stages: planning the review, conducting the review, and analyzing the results. In the planning stage, the research team defined the research questions (RQs) and formulated the overall strategy to ensure comprehensive coverage of the literature related to Large Language Models (LLMs) in Software Engineering (SE). The conducting stage involved a rigorous search and selection

process designed to identify relevant studies while minimizing bias and omissions. Finally, the analysis phase focused on synthesizing the findings from the selected studies to draw meaningful conclusions and insights into the use of LLMs for SE tasks.

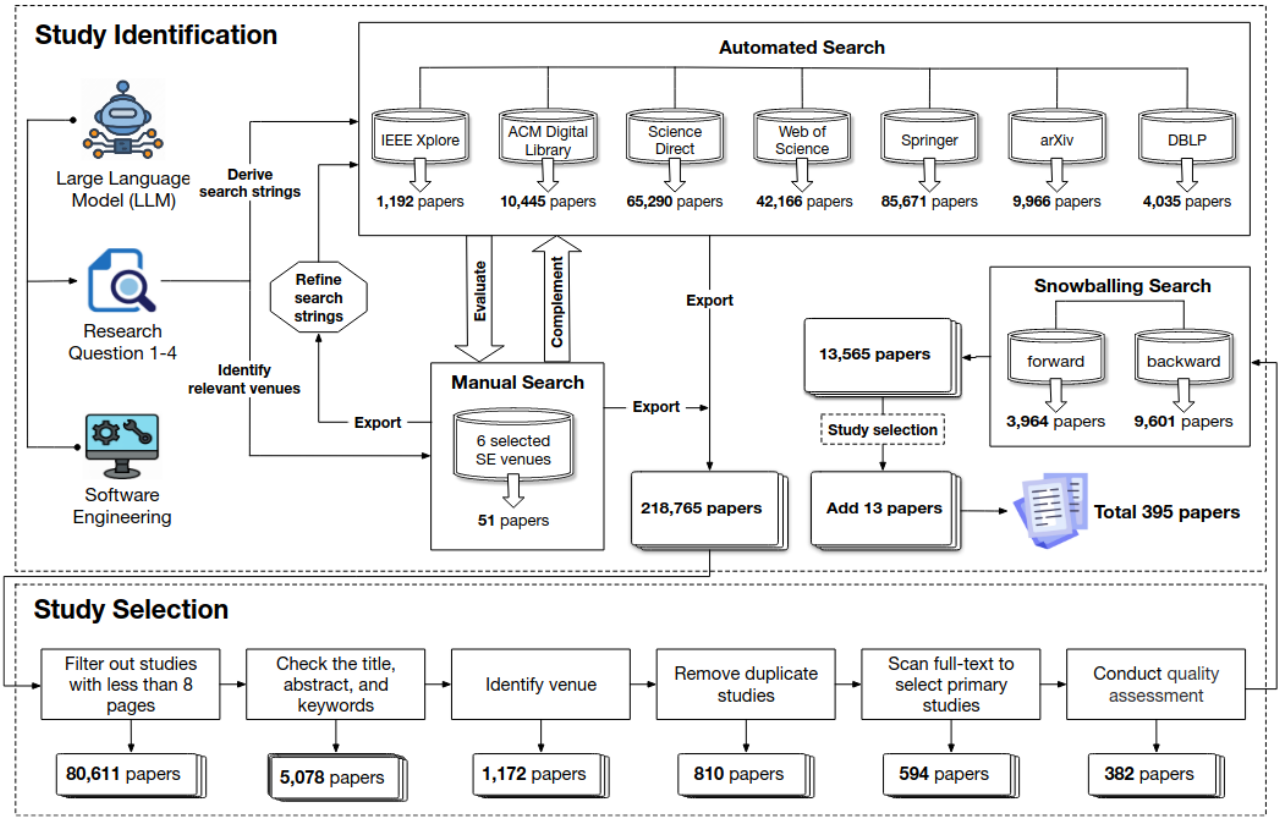


Figure 5: Study identification and selection process.

To identify relevant literature, a hybrid search strategy combining manual and automated searches was adopted as depicted in Figure 5. The team first employed the Quasi-Gold Standard (QGS) method, starting with a manual search across six premier SE conferences and journals—ICSE, ASE, ESEC/FSE, ISSTA, TOSEM, and TSE. From this manual search, a set of relevant seed papers was identified, forming the QGS base. Using this base, a comprehensive search string was constructed that integrated two categories of keywords: one set related to Software Engineering tasks (e.g., code generation, bug detection, program repair) and another pertaining to Large Language Models and related machine learning techniques (e.g., GPT, BERT, transformer, pre-trained model). This search string was then applied to

seven major digital libraries, including IEEE Xplore, ACM Digital Library, ScienceDirect, Web of Science, Springer, arXiv, and DBLP. The search covered literature published from January 2017 to January 2024, aligning with the emergence of the Transformer architecture, the foundation of modern LLMs.

Inclusion criteria	
1)	The paper claims that an LLM is used.
2)	The paper claims that the study involves an SE task.
3)	The paper with accessible full text.
Exclusion criteria	
1)	Short papers whose number of pages is less than 8.
2)	Duplicate papers or similar studies with different versions from the same authors.
3)	Studies belonging to books, thesis, monographs, keynotes, panels, or venues not executing a full peer-review process.
4)	Tool demos and editorials.
5)	The paper is published in a workshop or a doctoral symposium.
6)	The paper is a grey publication, e.g., a technical report or thesis.
7)	Non-English written literature.
8)	The paper mentions the use of LLMs without describing the employed techniques.
9)	The paper leverages SE methods to enhance LLMs, rather than focusing on using LLMs for SE tasks.

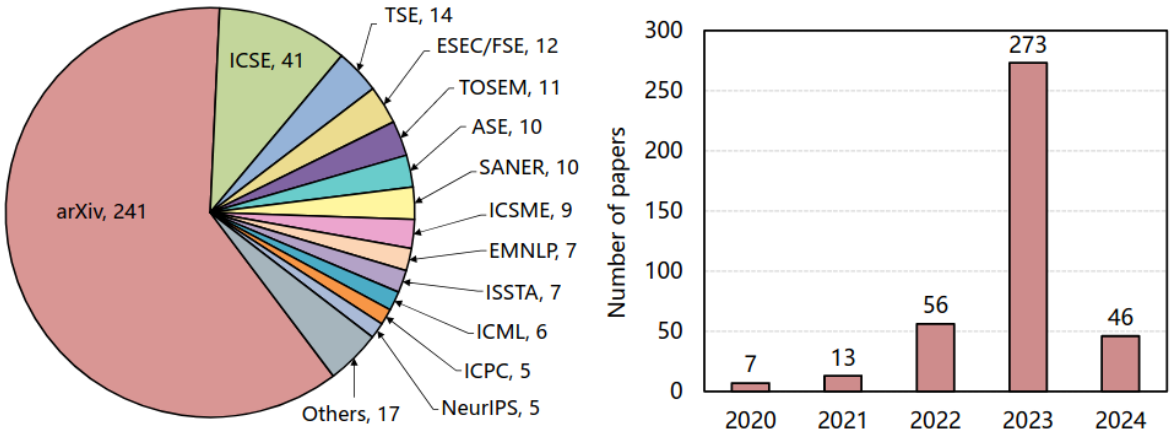
Figure 6: Inclusion criteria and Exclusion criteria.

The initial search produced over 218,000 articles, which were systematically filtered using inclusion and exclusion criteria to ensure relevance and quality as shown in Figure 6. Criteria included the explicit use of LLMs in SE tasks, full-text accessibility, and publication in reputable venues, while studies such as theses, editorials, tool demos, and non-peer-reviewed reports were excluded. Quality assessment was conducted using a ten-point checklist focusing on clarity, methodology, experimental setup, and contribution as illustrated in Figure 7. Articles were scored accordingly, with only high-quality studies retained. To further expand the dataset and ensure completeness, forward and backward snowballing was performed using citation tracking on the remaining articles, resulting in additional relevant studies.

ID	Quality Assessment Criteria
QAC1	Is the study relevant to SE tasks?
QAC2	Does the study utilize LLMs?
QAC3	Is the research not a secondary study, such as an SLR, review, or survey?
QAC4	Was the research published in a high-repute venue?
QAC5	Is there a clear motivation for the research?
QAC6	Does the study provide a clear description of the techniques used?
QAC7	Are the experimental setups, including experimental environments and dataset information, described in detail?
QAC8	Does the study clearly confirm the experimental findings?
QAC9	Are the key contributions and limitations of the study discussed?
QAC10	Does the study make a contribution to the academic or industrial community?

Figure 7: Checklist of Quality Assessment Criteria (QAC) for LLM studies in SE.

The final dataset comprised 395 studies published between 2017 and 2024 as shown in Figure 8. Data extraction was then carried out to capture essential information aligned with the research questions, including the types of LLMs used, dataset characteristics, optimization and evaluation methods, and the specific SE tasks addressed. The analysis phase synthesized these findings to identify patterns, trends, and gaps in the existing literature. This structured and transparent approach ensured both the comprehensiveness and validity of the review, providing a robust foundation for understanding the evolving role of Large Language Models in Software Engineering research.



(a) Distribution of papers across venues. (b) Distribution of papers over years.

Figure 8: Overview of the selected 395 papers' distribution.

Limitations and Future Directions

Despite the comprehensiveness of this study, several limitations must be acknowledged. First, although the search strategy was designed to be exhaustive by combining manual, automated, and snowballing techniques, the selection of search keywords and databases may still have led to the omission of some relevant studies. In particular, the reliance on English-language publications could have excluded potentially valuable research written in other languages. Furthermore, the dynamic nature of the field of Large Language Models (LLMs) means that new studies are continuously emerging. As a result, some of the most recent advancements published after the cut-off date of January 2024 might not be captured within this review. This temporal limitation highlights the rapidly evolving pace of LLM research and the challenge of maintaining a fully up-to-date synthesis.

Another limitation lies in the heterogeneity of the included studies. The reviewed works employed diverse datasets, architectures, and evaluation metrics, which posed difficulties in maintaining a consistent framework for comparison and synthesis. Although this review attempted to categorize models and applications systematically, the inherent differences in experimental setups and reporting standards limited the ability to perform a direct quantitative meta-analysis. Additionally, while quality assessment criteria were applied to

ensure the reliability of selected articles, such assessments inevitably involve a degree of subjectivity. Variations in reviewer interpretation or judgment could introduce minor biases, despite efforts to ensure inter-reviewer consistency.

Future work can address these limitations in several ways. First, extending the review with a living systematic review framework would enable continuous updates as new research emerges, ensuring that the synthesis remains current with ongoing developments in LLM architectures and applications. Second, future studies could explore quantitative meta-analysis techniques to provide stronger statistical evidence of performance trends and correlations between LLM types, optimization methods, and Software Engineering (SE) task performance. Third, a deeper investigation into industrial and proprietary datasets is needed, as most existing studies rely heavily on open-source data. Incorporating real-world datasets from industry could help assess the external validity and scalability of LLM-based SE tools in practical development environments.

Moreover, future research should focus on standardizing evaluation protocols to facilitate more consistent benchmarking across studies. Establishing common metrics, benchmark datasets, and reproducibility practices will enhance comparability and transparency in LLM4SE research. Another promising direction is exploring cross-disciplinary integration, such as combining insights from cognitive science, human-computer interaction, and ethics to better understand and govern the human–AI collaboration process in SE. Finally, as LLMs continue to grow in size and complexity, attention should be given to ethical, interpretability, and sustainability challenges, including model transparency, bias mitigation, and energy efficiency. Addressing these issues will not only advance the scientific understanding of LLMs in SE but also ensure their responsible and sustainable deployment in real-world applications.

Why it is relevant to my research?

LLMs are continuously evolving, and their applications are furthering the boundaries of computer science. Our objective is to fine-tune one of the suitable models to perform Android application-relevant tasks such as resource leaks.

BIBLIOGRAPHY

BIBLIOGRAPHY

[1] T. Liu, W. Ji, X. Dong, W. Yao, Y. Wang, H. Liu, H. Peng, and Y. Wang, "Jleaks: A featured resource leak repository collected from hundreds of open-source java projects," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, (New York, NY, USA), Association for Computing Machinery, 2024.

[2] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.

[3] F. Richter, "Charted: There are more mobile phones than people in the world," April 2023.

[4] Google Android Team, "Android — do more with google on android phones & devices." https://www.android.com/intl/en$_u s$/, 2025.

[5] Apple Inc., "iOS 26." https://www.apple.com/os/ios/, 2025. Accessed: 2025-10-16.

[6] P. Arntz, "Android threats rise sharply, with mobile malware jumping by 151% since start of year." Malwarebytes Blog, June 2025. Accessed on 16 October 2025.

[7] S. L. Sanna, D. Soi, D. Maiorca, G. Fumera, and G. Giacinto, "A risk estimation study of native code vulnerabilities in android applications," *Journal of Cybersecurity*, vol. 10, no. 1, p. tyae015, 2024.

[8] Android Developers, "Overview of memory management," 2023. Last updated: May 9, 2023; Accessed: October 17, 2025.

[9] Google Maps, "Google maps location or data." https://developers.google.com/maps/documentation/pl
sdk/memory-best-practices. Accessed October 17, 2025.

[10] B. Li, Q. Zhao, S. Jiao, and X. Liu, "Droidperf: Profiling memory objects on android devices," in *Proceedings of the 29th Annual International Conference on Mobile Com-*

puting and Networking, ACM MobiCom '23, (New York, NY, USA), Association for Computing Machinery, 2023.

[11] H. Zhang, D. She, and Z. Qian, "Android ion hazard: The curse of customizable memory management system," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1663–1674, 2016.

[12] Y. Liu, J. Wang, L. Wei, C. Xu, S.-C. Cheung, T. Wu, J. Yan, and J. Zhang, "Droidleaks: a comprehensive database of resource leaks in android apps," *Empirical Software Engineering*, vol. 24, no. 6, pp. 3435–3483, 2019.

[13] M. Ghanavati, D. Costa, J. Seboek, D. Lo, and A. Andrzejak, "Memory and resource leak defects and their repairs in java projects," *Empirical Software Engineering*, vol. 25, no. 1, pp. 678–718, 2020.

[14] "SpotBugs: Find bugs in Java Programs." https://spotbugs.github.io/, 2024.

[15] Facebook, "Infer Static Analyzer." https://github.com/facebook/infer, 2025.

[16] M. A. Akbar, A. A. Khan, and P. Liang, "Ethical aspects of chatgpt in software engineering research," *IEEE Transactions on Artificial Intelligence*, 2023.

[17] S. Wang, L. Huang, A. Gao, J. Ge, T. Zhang, H. Feng, I. Satyarth, M. Li, H. Zhang, and V. Ng, "Machine/deep learning for software engineering: A systematic literature review," *IEEE Transactions on Software Engineering*, vol. 49, no. 3, pp. 1188–1231, 2022.

[18] J. Cao, M. Li, M. Wen, and S.-c. Cheung, "A study on prompt design, advantages and limitations of chatgpt for deep learning program repair," *Automated Software Engineering*, vol. 32, no. 1, pp. 1–29, 2025.

[19] N. Tihanyi, Y. Charalambous, R. Jain, M. A. Ferrag, and L. C. Cordeiro, "A new era in software security: Towards self-healing software via large language models and formal verification," in *2025 IEEE/ACM International Conference on Automation of Software Test (AST)*, pp. 136–147, IEEE, 2025.

[20] M. Lajkó, V. Csuvik, and L. Vidács, "Towards javascript program repair with generative pre-trained transformer (gpt-2)," in *Proceedings of the third international workshop on automated program repair*, pp. 61–68, 2022.

[21] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[22] R. OpenAI, "Gpt-4 technical report. arxiv 2303.08774," *View in Article*, vol. 2, no. 5, p. 1, 2023.

[23] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Transactions on Software Engineering*, vol. 50, no. 4, pp. 911–936, 2024.

[24] D. Zan, B. Chen, F. Zhang, D. Lu, B. Wu, B. Guan, Y. Wang, and J.-G. Lou, "Large language models meet nl2code: A survey," *arXiv preprint arXiv:2212.09420*, 2022.

[25] G. Sridhara, S. Mazumdar, *et al.*, "Chatgpt: A study on its utility for ubiquitous software engineering tasks," *arXiv preprint arXiv:2305.16837*, 2023.

[26] Y. Liu, C. Tantithamthavorn, L. Li, and Y. Liu, "Deep learning for android malware defenses: a systematic literature review," *ACM Computing Surveys*, vol. 55, no. 8, pp. 1–36, 2022.